МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
САМАРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Институт автоматики и информационных технологий
Кафедра «Информатика и вычислительная техника»

ОТЧЕТ

по лабораторной работе № 3

по дисциплине Программирование микропроцессоров в технических системах

на тему Прерывания и таймеры

Преподаватель			12.10.26	Хмелевской А. А.
	(должность)	(подпись)	(дата)	(инициалы, фамилия)
Студенты			12.10.26	Латыпов Е. А.
	(группа)	(подпись)	(дата)	(инициалы, фамилия)
	(группа)	(подпись)	(дата)	(инициалы, фамилия)

Самара 2026

Цель: изучить аппаратные прерывания Arduino (attachInterrupt) и прерывания таймеров (Timer1, Timer2 в режиме CTC), и на их основе реализовать мигание светодиода на пине 13 с управлением по кнопке.

Без прерываний программа должна постоянно опрашивать кнопку в `loop()` (`digitalRead`) и может пропустить короткое нажатие, если занята другим делом. С прерыванием всё наоборот: когда сигнал на пине меняется, процессор сам останавливает основную программу, выполняет короткую функцию-обработчик и возвращается обратно. Поэтому нажатие не теряется, а `loop()` не тратит время на опрос. В Arduino Uno внешние прерывания есть только на пинах 2 (INT0) и 3 (INT1).

Код по частям

```
const int PIN_BUTTON = 2;
volatile bool pressed = false;
volatile unsigned long lastPress = 0;
unsigned long count = 0;
```

- `PIN_BUTTON = 2` — кнопка на пине с прерыванием.
- `pressed` — флаг «нажатие было». Обработчик его ставит, а `loop()` читает. Переменные, которые меняются в обработчике, объявляются как `volatile`, иначе компилятор может «оптимизировать» их и `loop()` не заметит изменения.
- `lastPress` — время последнего принятого нажатия, нужно для антидребезга.
- `count` — сколько нажатий поймали, только для вывода в консоль.

```
void onButton() {
  if (millis() - lastPress >= 50) {
    lastPress = millis();
    pressed = true;
  }
}
```

Это обработчик прерывания. Он короткий и делает только два дела: проверяет, что с прошлого нажатия прошло не меньше 50 мс (иначе это дребезг контактов), и ставит флаг. Внутри нет `delay()` и `Serial`, это запрещено в обработчиках из-за того, что они блокируют остальные прерывания.

```
void setup() {
```

```
  Serial.begin(9600);
  pinMode(PIN_BUTTON, INPUT_PULLUP);

attachInterrupt(digitalPinToInterrupt(PIN_BUTTON),
onButton, FALLING);
}
```

- `INPUT_PULLUP` включает внутреннюю подтяжку к питанию: без нажатия на пине HIGH, при нажатии кнопка замыкает его на GND и получается LOW.
- `attachInterrupt(номер_прерывания, функция, режим)` связывает пин с обработчиком. `digitalPinToInterrupt(2)` переводит номер пина в номер прерывания (это то же самое, что `0` в примере из методички).
- `FALLING` — сработать при переходе с HIGH на LOW, то есть в момент нажатия. Другие режимы: `RISING` (с LOW на HIGH), `CHANGE` (при любом изменении), `LOW` (пока на пине LOW).

```
void loop() {
  if (pressed) {
    pressed = false;
    count++;
    Serial.print("Interrupt! press #");
    Serial.println(count);
  }
}
```

`loop()` проверяет флаг. Если нажатие было, он сбрасывает флаг, увеличивает счётчик и печатает строку в Serial Monitor. Печать происходит здесь, а не в обработчике, и это сознательно.

Обычно мигание делают через `delay()` или через проверку `millis()` в `loop()`. Здесь мигание делает таймер процессора: он считает такты сам, независимо от программы, и когда досчитал до нужного числа, вызывает прерывание. Обработчик прерывания переключает светодиод. `loop()` при этом пустой, и в нём можно делать что угодно, мигание не собьётся.

Код

```
const int PIN_LED = 13;

ISR(TIMER1_COMPA_vect) {
  digitalWrite(PIN_LED, !digitalRead(PIN_LED));
}
```

`ISR(TIMER1_COMPA_vect)` — обработчик прерывания: «Timer1, совпадение с регистром A». Он вызывается автоматически раз в секунду и переключает светодиод на противоположное состояние: горел, значит гаснет, и наоборот.

```
void setup() {
  pinMode(PIN_LED, OUTPUT);
  cli();
  TCCR1A = 0;
  TCCR1B = 0;
  OCR1A = 15624;
  TCCR1B |= (1 << WGM12);
  TCCR1B |= (1 << CS10) | (1 << CS12);
  TIMSK1 |= (1 << OCIE1A);
  sei();
}
```

Настройка таймера по строкам:

- `cli()` — на время настройки выключить прерывания, чтобы таймер не сработал на полпути. `sei()` в конце включает их обратно.
- `TCCR1A = 0; TCCR1B = 0;` — обнулить управляющие регистры Timer1, чтобы не мешали старые настройки.
- `OCR1A = 15624` — число, до которого считает таймер. Дойдя до него, он обнуляется и вызывает прерывание.
- `TCCR1B |= (1 << WGM12)` — режим CTC («сброс по совпадению»): таймер считает не до переполнения, а до значения в `OCR1A`.

- `TCCR1B |= (1 << CS10) | (1 << CS12)` — делитель частоты 1024. Это биты выбора тактового источника из таблицы в методичке.
- `TIMSK1 |= (1 << OCIE1A)` — разрешить прерывание по совпадению, иначе таймер считает, но обработчик не вызывается.

```
void loop() {
}
```

Пустой: всё делает прерывание.

Откуда число 15624

Процессор работает на 16 МГц. С делителем 1024 таймер получает 16 000 000 / 1024 = 15625 тактов в секунду. Чтобы отсчитать 1 секунду, нужно 15625 тактов. В режиме CTC сброс счётчика в ноль занимает один такт, поэтому в регистр пишут на единицу меньше: 15624 (этот расчёт есть в методичке).

Важная деталь

Светодиод переключается раз в секунду, значит секунду горит и секунду не горит, период мигания 2 секунды (0,5 Гц). Это то, о чём вы просили. Для частоты мигания 1 Гц (по заданию пункта 3) переключать нужно каждые 0,5 с, и тогда в `OCR1A` ставится 7811. Поэтому в `blink_cycle.ino` стоит именно это значение.

Здесь три прерывания работают вместе, а `loop()` занимается только печатью:

- **Кнопка (пин 2)** запускает цикл и перезапускает его при каждом нажатии.
- **Timer1** переключает светодиод каждые 0,5 с. Это даёт мигание 1 Гц.
- **Timer2** отсчитывает 5 секунд и переключает фазу: мигание на паузу и обратно, по кругу.

Переменные

```
volatile bool running = false;      // цикл запущен кнопкой
volatile bool blinkPhase = true;   // true - мигание, false - пауза
volatile bool pressed = false;     // флаг для вывода в консоль
volatile unsigned int ticks = 0;   // счётчик тиков Timer2
unsigned long lastPress = 0;             // для антидребезга
unsigned long prevPress = 0;         // для вывода времени между нажатиями
```

Все переменные, которые меняются в прерываниях, объявлены `volatile`. Пока кнопку не нажали, `running` равно `false`, и светодиод не горит.

Прерывание Timer1 (мигание)

```
ISR(TIMER1_COMPA_vect) {
  if (running && blinkPhase) digitalWrite(PIN_LED, !digitalRead(PIN_LED));
}
```

Срабатывает каждые 0,5 с, но светодиод переключает только если цикл запущен и сейчас фаза мигания. В паузе оно ничего не делает.

Прерывание Timer2 (5 секунд)

```
ISR(TIMER2_COMPA_vect) {
  if (running && ++ticks >= 1250) {
    ticks = 0;
    blinkPhase = !blinkPhase;
```

```
      digitalWrite(PIN_LED, LOW);
    }
  }
```

Timer2 восьмибитный и не может отсчитать 5 секунд сразу, поэтому он срабатывает каждые 4 мс, а обработчик считает такты. 1250 тактов по 4 мс дают ровно 5 секунд. После этого счётчик обнуляется, фаза меняется на противоположную, светодиод гасится. Дальше его мигание ведёт Timer1.

Прерывание кнопки

```
void onButton() {
  if (millis() - lastPress >= 50) {
    lastPress = millis();
    running = true;
    blinkPhase = true;
    ticks = 0;
    TCNT1 = 0;
    TCNT2 = 0;
    digitalWrite(PIN_LED, HIGH);
    pressed = true;
  }
}
```

Сначала антидребезг на 50 мс. Затем цикл запускается или перезапускается с начала: фаза мигания, счётчик тактов в ноль, оба таймера обнуляются (`TCNT1`, `TCNT2` — их счётчики), чтобы первые 0,5 с и 5 с отсчитывались полностью. Светодиод загорается сразу, а флаг `pressed` уходит в `loop()` для вывода.

Настройка таймеров в `setup()`

```
  TCCR1B = (1 << WGM12) | (1 << CS12) | (1 << CS10);
// Timer1: CTC, делитель 1024
  OCR1A = 7811;                                    //
0,5 с
  TCCR2A = (1 << WGM21);                           //
Timer2: CTC
  TCCR2B = (1 << CS22) | (1 << CS21);              //
делитель 256
  OCR2A = 249;                                     //
4 мс
  TIMSK1 = (1 << OCIE1A);  TIMSK2 = (1 << OCIE2A);
// разрешить оба прерывания
```

Расчёты:

- Timer1: 16 МГц / 1024 = 15625 Гц. 0,5 с это 7812 отсчётов, в регистр пишем на единицу меньше: 7811.
- Timer2: 16 МГц / 256 = 62500 Гц. 4 мс это 250 отсчётов, в регистр пишем 249.

Всё остальное (`cli()` и `sei()`, режим CTC, `TIMSK`) работает так же, как в пункте 2.

`loop()`

```
if (pressed) {
  pressed = false;
  unsigned long now = millis();
  Serial.print("Time since last press, ms: ");
  Serial.println(now - prevPress);
  prevPress = now;
}
```

Если было нажатие, `loop()` печатает, сколько миллисекунд прошло с предыдущего. Для первого нажатия это время с запуска платы. Печать сделана здесь, а не в обработчике, потому что `Serial` внутри прерываний использовать нельзя.

Как это выглядит на практике

Нажали кнопку: светодиод мигает пять раз по секунде, потом на 5 секунд гаснет, потом снова мигает, и так по кругу. Нажали ещё раз в любой момент: всё сбрасывается и цикл начинается заново с мигания, а в консоли появляется время между двумя нажатиями.

Один нюанс: в начале второго и следующих циклов первая вспышка иногда сдвигается на полпериода (0,5 с). Это из-за того, что прерывания двух таймеров в момент смены фазы совпадают по времени. Пять миганий по 1 Гц и пять секунд паузы при этом сохраняются.

Вывод: реализованы три программы: обработка нажатия кнопки по внешнему прерыванию, мигание светодиода по прерыванию Timer1, и цикл «5 с мигает, 5 с выключен» по кнопке на двух таймерах. Прерывания позволяют реагировать на события мгновенно, а `loop()` при этом остаётся свободным. Для этого обработчики должны быть короткими, без `delay()` и `Serial`, а общие переменные объявляются как `volatile`. Таймеры считают время независимо от программы, а для интервалов, которые не умещаются в один таймер (например, 5 с на 8-битном Timer2), используют счётчик тактов в обработчике. Дребезг кнопки подавляется программно через `millis()`.